

4. Neural Networks and Backpropagation (1)

흐름: 이전 강의 복습 → Deep Learning 사례 → Neural Networks → Gradient 계산 문제 → Computational Graph → Backpropagation → Vector-Matrix Backpropagation

1. Neural Networks
2. Backpropagation

1. Neural Networks (FCN)

neural network = fully-connected network = multi-layer perceptrons

1.1 Linear Classifier → Neural Network

기존 Linear Classifier: 입력 → 출력까지 선형 변환만 진행

Neural Network: 비선형성 추가

[Non-linearity가 필요한 이유]

- 원래 공간: 하나의 직선으로 분리 불가 ⇒ 변환된 공간: 선형 분류기로 분리 가능
- Neural network의 hidden layer: feature transformation을 학습.

1.2 Fully Connected Network

각 layer의 모든 입력이 다음 layer의 모든 neuron과 연결됨.

- Linear Classifier: $f = Wx$
- 2-layer network: $f = W_2 \max(0, W_1 x)$
- 3-layer network: $f = W_3 \max(0, W_2 \max(0, W_1 x))$

(예시) 2-layer network의 계산 과정:

$$x \xrightarrow{W_1} h \xrightarrow{\max(0, \cdot)} h \xrightarrow{W_2} s$$

- Linear classifier: class별로 1개씩, 총 10개의 template 학습.

- 2-layer network: W_1 을 통해 100개의 template을 먼저 학습. W_2 는 이 template들을 조합하여 class score를 계산.
 - 여러 class가 중간 template 공유 가능.
 - layer가 깊어질수록 이전 layer에서 만든 feature를 조합해 더 높은 수준의 표현 구성.

1.3 Activation Function

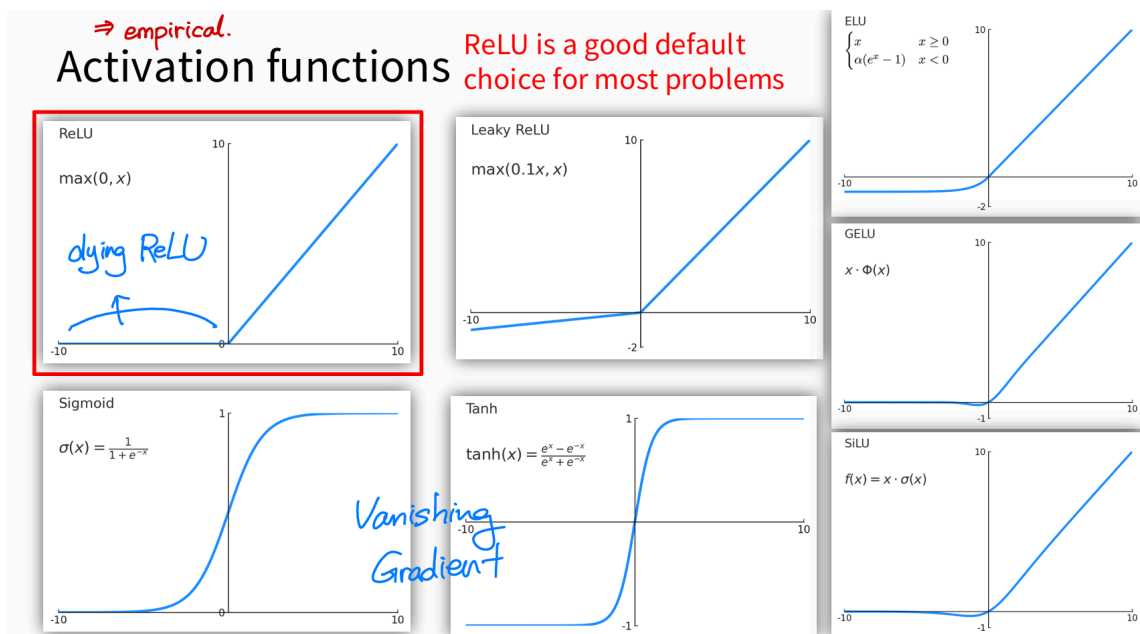
Activation function: layer 사이의 non-linearity 부여

→ 여러 선형 변환을 쌓아도 하나의 선형 변환과 같음

Activation function 종류

⇒ 전부 empirical

▼ 사진



- **ReLU:** $f(x) = \max(0, x)$

가장 보편적. dying ReLU 발생(음수 영역 → 출력, gradient = 0. neuron 이 계속 비활성화).

- **Leaky ReLU:** $f(x) = \max(0.1x, x)$

- **Tanh:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- **ELU:** $f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$

- **GELU:** $f(x) = x\Phi(x)$

- **SiLU:** $f(x) = x\sigma(x)$

음수 영역에도 작은 기울기를 둬.

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$

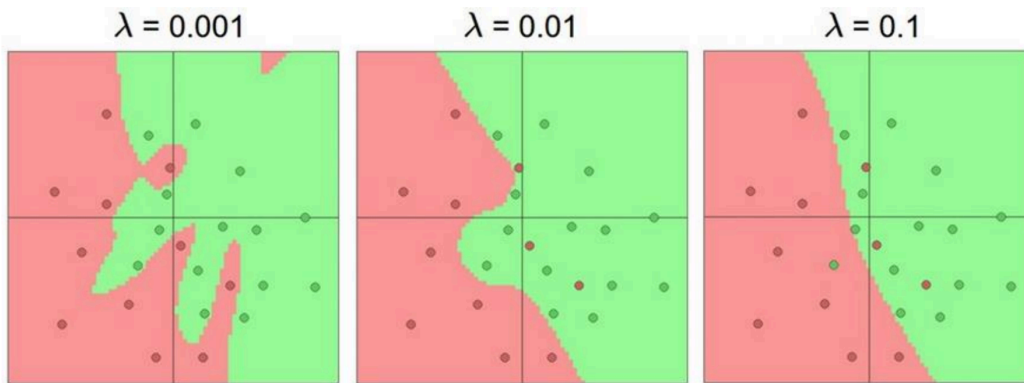
vanishing gradient 발생 (큰 양수나 음수 영역 → gradient가 매우 작아짐)

1.4 Layer 크기, Model Capacity

hidden neuron(layer)이 많아질수록, 더 복잡한 decision boundary 표현 가능.

주의: network 크기 자체를 regularizer로 사용 X.

⇒ 작은 network로 표현력을 제한하기보다, 충분한 capacity를 두고 regularization 강도 조절.



$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W)$$

- 작은 λ : 복잡한 decision boundary. overfitting 가능성 증가
- 큰 λ : 더 부드러운 decision boundary

2. Gradient 계산 방식

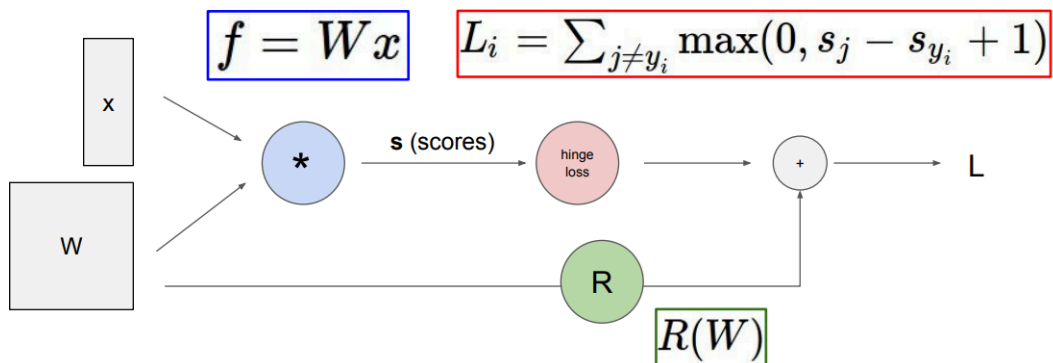
score function, loss function, regularization, total loss를 전부 전개하기 현실적으로 어려움.

⇒ **computational graph + backpropagation** 사용.

- Forward pass: graph 입력 → 출력 계산
- Backward pass: 최종 loss → 입력 계산. gradient 전달 목적.

2.1 Computational Graph

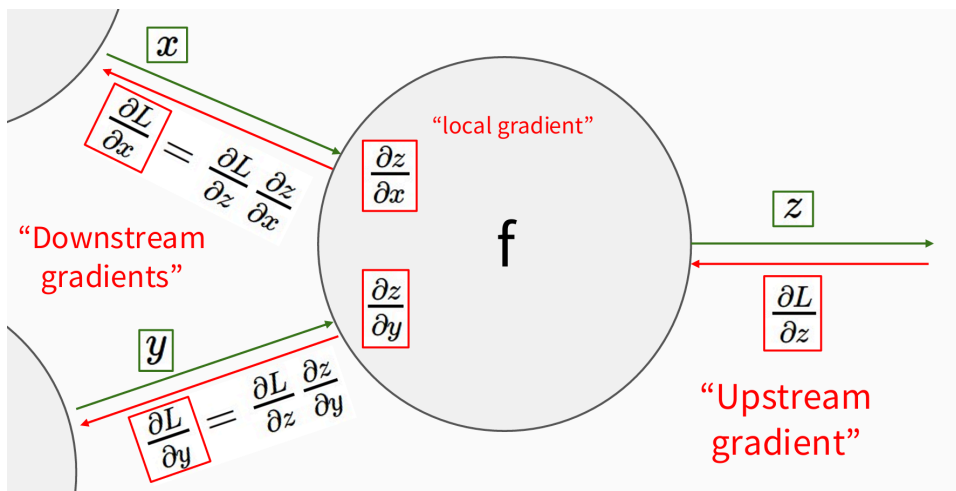
Computational graphs



2.2 Backpropagation

computation graph에서 **Chain Rule** 적용

⇒ Upstream gradient → Downstream gradient 전달 가능



(예시 1)

주어진 값

계산

Backpropagation: a simple example

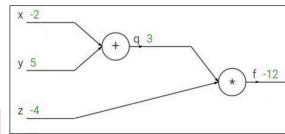
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: a simple example

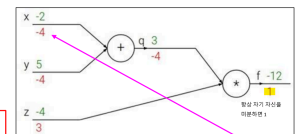
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream gradient Local gradient

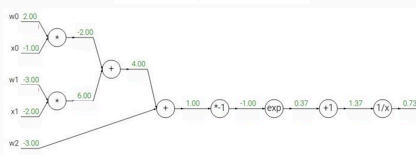
(예시 2)

주어진 값

계산

Another example:

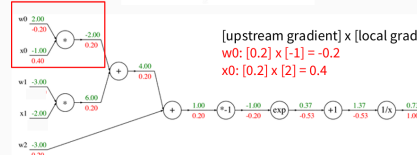
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



$$\begin{array}{ll} f(x) = e^x & \rightarrow \frac{df}{dx} = e^x \\ f_a(x) = ax & \rightarrow \frac{df}{dx} = a \end{array} \quad \begin{array}{ll} f(x) = \frac{1}{x} & \rightarrow \frac{df}{dx} = -1/x^2 \\ f_c(x) = c + x & \rightarrow \frac{df}{dx} = 1 \end{array}$$

Another example:

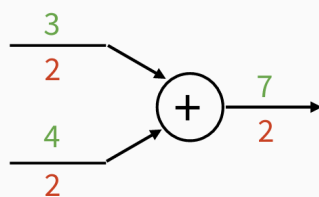
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



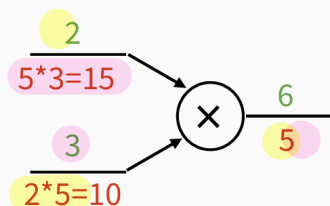
$$\begin{array}{ll} f(x) = e^x & \rightarrow \frac{df}{dx} = e^x \\ f_a(x) = ax & \rightarrow \frac{df}{dx} = a \end{array} \quad \begin{array}{ll} f(x) = \frac{1}{x} & \rightarrow \frac{df}{dx} = -1/x^2 \\ f_c(x) = c + x & \rightarrow \frac{df}{dx} = 1 \end{array}$$

2.3 Gradient Flow 기본 패턴

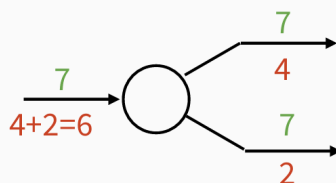
add gate: gradient distributor



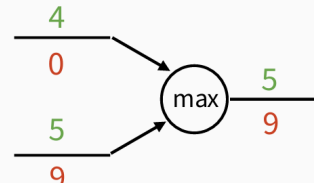
mul gate: "swap multiplier"



copy gate: gradient adder



max gate: gradient router



- Add Gate: Upstream gradient를 두 입력에 그대로 전달.
- Multiply Gate: 각 입력의 gradient에 반대쪽 입력값을 곱함.
- Copy Gate: 하나의 값이 여러 branch에서 사용되면 각 branch에서 돌아온 gradient를 더함.

- Max Gate: Forward pass에서 선택된 입력에만 gradient 전달. 선택되지 않은 입력의 gradient는 0.
-